

Python pour le trading — Document 5/8

La manipulation de données

Fil conducteur. Toujours le gestionnaire d'ordres de trading. Ce document est ton terrain : ton expérience en apprentissage automatique te donne déjà une bonne base ici. Manipuler efficacement des données de marché — prix, volumes, séries temporelles — est au cœur du métier. On y voit NumPy (le calcul rapide), Pandas (les tableaux de données) et la gestion délicate du temps.

Comment lire ce document. Chaque section présente d'abord l'idée en mots simples, puis un exemple commenté, puis une explication. Le texte entre les blocs de code porte l'essentiel : ne le saute pas.

1. NumPy : le calcul vectorisé

NumPy fournit l'**array**, un tableau de nombres bien plus rapide qu'une liste Python pour le calcul. Son secret est la **vectorisation** : appliquer une opération à tout le tableau d'un coup, au lieu de boucler élément par élément. C'est la base de toute la pile data de Python.

1.1 La vectorisation

```
import numpy as np

prix = np.array([5230.0, 5231.5, 5232.0, 5233.25])

# SANS vectorisation (boucle Python, lent)
resultat = []
for p in prix:
    resultat.append(p * 1.01)

# AVEC vectorisation (une seule opération, rapide)
resultat = prix * 1.01      # applique * 1.01 à TOUT le tableau d'un coup
# -> array([5282.3, 5283.815, 5284.32, 5285.5825])
```

Avec NumPy, `prix * 1.01` multiplie chaque élément sans qu'on écrive de boucle. Ce n'est pas qu'une question de concision : sous le capot, l'opération est exécutée en code C optimisé, donc bien plus vite qu'une boucle Python. Sur des millions de prix, l'écart de vitesse est considérable. La règle d'or de NumPy : éviter les boucles Python, penser en opérations sur des tableaux entiers.

1.2 Vectorisation, code natif et GIL : le lien avec le document 4

Tu fais exactement le bon rapprochement avec le document 4, et la réponse est **oui**, avec une nuance utile. La vectorisation NumPy délègue le calcul à du **code natif** (C/Fortran, via des bibliothèques comme BLAS, MKL ou OpenBLAS). Cela apporte **deux** gains distincts qu'il faut bien séparer :

1. On évite l'interpréteur Python. Même sur un seul cœur, le code C est intrinsèquement plus rapide qu'une boucle Python, car il ne re-vérifie pas le type à chaque élément. C'est la source principale du gain (souvent un facteur 10 à 100).

2. Le code natif relâche le GIL. Pendant qu'il s'exécute, ce code C libère le verrou global (le GIL, vu au document 4). Cela permet, pour les opérations lourdes (typiquement l'algèbre matricielle comme $A @ B$), de **paralléliser sur plusieurs cœurs** à l'intérieur de la bibliothèque. Une boucle Python pure, elle, garderait le GIL et resterait mono-cœur.

***Pourquoi ça compte.** La nuance : toutes les opérations NumPy ne sont pas multi-cœurs. Les opérations simples élément par élément (prix * 1.01) sont surtout accélérées par le point 1 (code C, mono-cœur). Les grosses opérations d'algèbre linéaire (multiplication de matrices) profitent en plus du point 2 (multi-cœurs via BLAS). Dans les deux cas, on est plus rapide qu'une boucle Python ; mais « libérer le GIL » et « être plus rapide » ne sont pas tout à fait la même chose. Le premier réflexe vitesse reste la vectorisation, indépendamment du nombre de cœurs.*

À retenir : vectoriser, c'est d'abord sortir de l'interpréteur Python (toujours un gain) ; et, pour les calculs lourds, c'est aussi profiter de plusieurs cœurs car le code natif relâche le GIL. Ton intuition est juste, il faut seulement distinguer ces deux effets.

Pourquoi ça compte. Calculer des rendements, des moyennes mobiles ou des indicateurs sur des millions de barres en boucle Python serait trop lent. La vectorisation NumPy rend ces calculs quasi instantanés, ce qui est indispensable pour la recherche quantitative.

1.3 Opérations utiles

```
prix = np.array([5230.0, 5231.5, 5232.0, 5233.25, 5234.50])
```

```
prix.mean()           # moyenne -> 5232.25
prix.std()             # écart-type (volatilité)
prix.max() - prix.min() # amplitude (range)
np.diff(prix)          # différences successives -> rendements bruts
prix[prix > 5232]       # filtre booléen : garde les prix > 5232
```

NumPy offre des opérations statistiques directes (mean, std...) et le **filtrage booléen** : `prix[prix > 5232]` renvoie tous les prix supérieurs à 5232, sans boucle ni condition explicite. `np.diff` calcule les différences successives — la base du calcul des rendements. Ces opérations vectorisées sont les briques de toute analyse de marché.

Les opérations NumPy importantes en un coup d'œil :

Opération	Ce qu'elle fait
<code>np.array(liste)</code>	Crée un array NumPy à partir d'une liste Python.

Opération	Ce qu'elle fait
<code>a.mean()</code>	Moyenne des éléments.
<code>a.std()</code>	Écart-type (mesure de dispersion ; en finance, la volatilité).
<code>a.sum()</code>	Somme de tous les éléments.
<code>a.min()</code> / <code>a.max()</code>	Plus petite / plus grande valeur.
<code>a.max() - a.min()</code>	Amplitude (l'étendue entre le min et le max).
<code>np.diff(a)</code>	Différences successives $a[i+1] - a[i]$; base des rendements bruts.
<code>a[a > seuil]</code>	Filtrage booléen : ne garde que les éléments vérifiant la condition.
<code>a * 1.01</code>	Opération vectorisée : applique le calcul à TOUT le tableau d'un coup.
<code>a @ b</code>	Produit matriciel ; opération lourde qui peut utiliser plusieurs cœurs.
<code>np.cumsum(a)</code>	Somme cumulée (utile pour un PnL cumulé, par exemple).
<code>a.reshape(1, c)</code>	Réorganise le tableau en 1 lignes et c colonnes.

Pourquoi ça compte. Ces quelques opérations couvrent la majeure partie des calculs de marché : rendements, volatilité, filtres, agrégats. Les maîtriser permet d'exprimer un indicateur en une ligne vectorisée plutôt qu'en une boucle lente.

2. Pandas : les DataFrames

Pandas bâtit sur NumPy et fournit le **DataFrame** : un tableau à colonnes nommées, comme une feuille de calcul ou une table de base de données. C'est l'outil de référence pour manipuler des données de marché structurées (date, symbole, prix, volume...).

2.1 Créer et explorer un DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "symbole": ["ES", "NQ", "ES", "CL"],
    "prix":    [5234.50, 18450.0, 5235.0, 78.20],
    "volume":  [1200, 800, 1500, 600],
})
```

```
df.head()           # aperçu des premières lignes
df["prix"]          # une colonne
df["prix"].mean()   # moyenne de la colonne prix
df.describe()       # statistiques résumées de toutes les colonnes
```

Un DataFrame se construit ici à partir d'un dictionnaire où chaque clé devient une colonne. On accède à une colonne par son nom (df["prix"]), et toutes les opérations NumPy s'y appliquent (.mean()). describe() donne d'un coup les statistiques de chaque colonne. C'est exactement le format dans lequel on charge des historiques de marché.

2.2 Exemples utiles au quotidien

Voici les manipulations les plus fréquentes en pratique, dont le chargement depuis un fichier (le point de départ de presque toute analyse).

Charger un fichier dans un DataFrame :

```
# Depuis un CSV (un relevé de courtier, un historique exporté)
df = pd.read_csv("historique_es.csv")

# Depuis un fichier Parquet (format compact pour gros volumes, voir doc 6)
df = pd.read_parquet("historique_es.parquet")

# Préciser qu'une colonne contient des dates, et en faire l'index
df = pd.read_csv("ticks.csv", parse_dates=["date"], index_col="date")
```

Sélectionner, ajouter, transformer :

```
df["prix"]           # une colonne (Series)
df[["symbole", "prix"]] # plusieurs colonnes (sous-DataFrame)
df.iloc[0]           # la première ligne (par position)
df.loc[df["symbole"] == "ES"] # les lignes ES (par condition)

# Ajouter une colonne calculée (vectorisé, sans boucle)
df["valeur"] = df["quantite"] * df["prix"]

# Statistiques rapides
df["prix"].describe() # count, mean, std, min, quartiles, max
df["symbole"].value_counts() # combien de lignes par symbole
```

2.3 Manipulations essentielles de nettoyage et de préparation

Avant d'analyser des données de marché, il faut presque toujours les **nettoyer et les préparer** : harmoniser les noms de colonnes, traiter les valeurs manquantes, normaliser les échelles, fusionner des sources. Ces opérations, familières en apprentissage automatique, sont au cœur du travail quotidien sur les données.

Renommer des colonnes

```
# Renommer certaines colonnes (par un dictionnaire ancien -> nouveau)
df = df.rename(columns={"px": "prix", "qty": "quantite"})
```

```
# Remplacer TOUS les noms de colonnes d'un coup
df.columns = ["symbole", "prix", "quantite", "date"]
```

```
# Normaliser les noms : minuscules, sans espaces superflus
df.columns = df.columns.str.strip().str.lower()
```

Le rename est essentiel quand on réconcilie deux sources aux conventions différentes (un courtier nomme « px », un autre « price ») : on harmonise avant de comparer. `df.columns.str.lower()` applique une transformation à tous les noms d'un coup.

Gérer les valeurs manquantes (NaN)

```
df.isna().sum()          # compter les valeurs manquantes par colonne
df = df.dropna()          # supprimer les lignes incomplètes
df = df.fillna(0)         # remplacer les manquantes par une valeur
df["prix"] = df["prix"].ffill() # propager la dernière valeur connue (forward fill)
```

Les données de marché ont souvent des trous (un tick manquant, une séance fériée). `dropna` supprime les lignes incomplètes ; `fillna` les comble ; `ffill` (forward fill) propage la dernière valeur connue — utile pour un prix qui n'a pas changé. Le choix dépend du contexte : supprimer un tick manquant n'a pas le même effet que reporter le dernier prix.

Normaliser et mettre à l'échelle

```
# Normalisation min-max : ramener une colonne dans l'intervalle [0, 1]
df["prix_norm"] = (df["prix"] - df["prix"].min()) / (df["prix"].max() - df["prix"].min())
```

```
# Standardisation (score z) : moyenne 0, écart-type 1
df["prix_z"] = (df["prix"] - df["prix"].mean()) / df["prix"].std()
```

```
# Rendements (variation relative d'une ligne à l'autre)
df["rendement"] = df["prix"].pct_change()
```

La **normalisation min-max** ramène les valeurs entre 0 et 1 ; la **standardisation** (score z) les centre sur une moyenne de 0 avec un écart-type de 1. Ce sont des étapes classiques avant d'alimenter un modèle, et tu les reconnaîtras de ton expérience en ML. `pct_change` calcule directement les rendements, plus parlants que les prix bruts pour comparer des actifs d'échelles différentes (ES à 5000, CL à 78).

Trier, dédupliquer, réindexer

```
df = df.sort_values("date")          # trier par date
df = df.drop_duplicates()            # retirer les lignes en double
df = df.reset_index(drop=True)       # renuméroter l'index proprement
df = df.set_index("date")            # faire d'une colonne l'index
```

Trier par date est indispensable pour une série temporelle. `drop_duplicates` retire les ticks reçus en double (fréquent avec un flux). `set_index("date")` transforme la colonne de dates en index, ce qui débloque ensuite `resample` et `rolling`.

Trier les valeurs à l'intérieur d'une ligne (sur certaines colonnes)

```
# Trier les VALEURS d'une ligne, sur un sous-ensemble de colonnes seulement.
# Ex : l'index est une date ; on veut trier les colonnes 2 à 4 par ligne,
#      sans toucher à la 5e colonne ni à l'index.
cols = df.columns[1:4]                # les colonnes d'indices 1, 2, 3
df[cols] = df[cols].apply(
    lambda ligne: sorted(ligne),       # sorted(ligne, reverse=True) -> décroissant
    axis=1,                            # axis=1 = ligne par ligne (et non colonne)
    result_type="expand",
)
```

Ici, `axis=1` est la clé : `apply` travaille **ligne par ligne** (horizontalement) au lieu de colonne par colonne. On ne sélectionne que les colonnes à trier (`cols`) : les autres colonnes et l'index de dates restent intacts. `result_type="expand"` rééte la liste triée sur les colonnes correspondantes. Subtilité importante : le tri réordonne les **valeurs**, mais elles restent rangées sous les mêmes noms de colonnes — la plus petite valeur se retrouve dans la première colonne de `cols`, etc. Les étiquettes de colonnes ne « suivent » donc pas les valeurs. (Si l'on voulait au contraire connaître quelle colonne porte la plus grande valeur par ligne, on utiliserait plutôt `idxmax(axis=1)`.)

Fusionner et concaténer des sources

```
# Empiler deux DataFrames de même structure (ex. deux journées)
df = pd.concat([df_lundi, df_mardi])

# Joindre deux sources sur une clé commune (ex. positions + relevé courtier)
fusion = pd.merge(positions, releve, on="symbole", how="inner")
```

`concat` empile des DataFrames (par exemple plusieurs journées d'historique). `merge` joint deux sources sur une colonne commune — exactement l'opération d'une réconciliation : rapprocher les positions internes et le relevé du courtier sur le symbole, puis comparer. C'est le pont vers la « résolution d'écarts » du métier.

Appliquer une transformation

```
# Transformer une colonne avec une fonction (à éviter si vectorisable)
df["cote"] = df["cote"].map({"B": "achat", "S": "vente"})
```

```
# apply pour une logique plus complexe (plus lent qu'une opération vectorisée)
df["categorie"] = df["volume"].apply(lambda v: "gros" if v > 1000 else "petit")
```

map remplace des valeurs selon une correspondance (par exemple normaliser des codes « B »/« S » en « achat »/« vente »). apply applique une fonction quelconque, mais reste plus lent qu'une opération vectorisée : on le réserve aux cas qui ne se vectorisent pas (rappel du document sur la performance).

Pourquoi ça compte. Le nettoyage et la préparation occupent souvent la majeure partie du temps d'un travail sur données. Renommer, combler les trous, normaliser et fusionner proprement, c'est ce qui sépare une analyse fiable d'un résultat faussé par des données mal alignées — un point critique pour la réconciliation et le reporting.

Les manipulations essentielles en un coup d'œil :

Opération	Ce qu'elle fait
<code>df.rename(columns={...})</code>	Renomme certaines colonnes (ancien -> nouveau).
<code>df.columns = [...]</code>	Remplace tous les noms de colonnes.
<code>df.isna().sum()</code>	Compte les valeurs manquantes par colonne.
<code>df.dropna()</code>	Supprime les lignes contenant des valeurs manquantes.
<code>df.fillna(v)</code>	Remplace les valeurs manquantes par v.
<code>.ffill()</code> / <code>.bfill()</code>	Propage la valeur précédente / suivante.
<code>(x - x.min()) / (x.max()-x.min())</code>	Normalisation min-max dans [0, 1].
<code>(x - x.mean()) / x.std()</code>	Standardisation (score z) : moyenne 0, écart-type 1.
<code>x.pct_change()</code>	Rendements : variation relative d'une ligne à l'autre.
<code>df.sort_values("col")</code>	Trie les lignes selon une colonne (croissant par défaut ; ascending=False pour décroissant).
<code>df.drop_duplicates()</code>	Retire les lignes en double.
<code>df[c].apply(sorted, axis=1, result_type="expand")</code>	Trie les valeurs À L'INTÉRIEUR de chaque ligne, sur les colonnes c.
<code>df.reset_index(drop=True)</code>	Renumérote l'index proprement.

Opération	Ce qu'elle fait
<code>df.set_index("col")</code>	Fait d'une colonne l'index (utile pour resample).
<code>pd.concat([a, b])</code>	Empile des DataFrames de même structure.
<code>pd.merge(a, b, on="cle")</code>	Joint deux sources sur une clé commune.
<code>df["c"].map({...})</code>	Remplace des valeurs selon une correspondance.
<code>df["c"].apply(fonction)</code>	Applique une fonction (à réserver au non-vectorisable).

Changer le type d'une colonne

Après un chargement, une colonne peut avoir le mauvais type : un nombre lu comme texte, un identifiant lu comme nombre, etc. Trier ou calculer sur le mauvais type donne des résultats faux (du texte se trie « 10 » avant « 2 »). La méthode principale est `astype`.

```
df["quantite"] = df["quantite"].astype(int)      # vers entier
df["prix"] = df["prix"].astype(float)           # vers nombre à virgule
df["symbole"] = df["symbole"].astype(str)       # vers texte
df["actif"] = df["actif"].astype(bool)          # vers booléen
```

```
# Convertir plusieurs colonnes d'un coup
df = df.astype({"quantite": int, "prix": float, "symbole": str})
```

```
# Vérifier les types avant / après
df.dtypes
```

`astype` convertit une colonne vers le type voulu (`int`, `float`, `str`, `bool`). On peut convertir plusieurs colonnes d'un coup en passant un dictionnaire. `df.dtypes` affiche le type de chaque colonne, à vérifier avant et après.

Pour les nombres mal formés. Si une valeur n'est pas convertible, `astype` plante. On utilise alors `pd.to_numeric` avec `errors="coerce"`, qui met NaN sur les valeurs illisibles au lieu de tout faire échouer :

```
df["prix"] = pd.to_numeric(df["prix"], errors="coerce")
# les valeurs non numériques deviennent NaN, on peut ensuite les traiter
```

Cas fréquent : virgule décimale (format européen). Un nombre écrit « 5234,50 » est vu comme du texte à cause de la virgule. Il faut d'abord remplacer la virgule par un point, puis convertir :

```
df["prix"] = df["prix"].str.replace(",", ".").astype(float)
```

On enchaîne `str.replace(",", ".")` (qui corrige le séparateur décimal) et `astype(float)` (qui convertit le texte corrigé en nombre). C'est un classique des fichiers de sources européennes.

Pourquoi ça compte. Le type est invisible mais décisif : une colonne de prix laissée en texte se trierait et se calculerait à tort. Le réflexe après tout chargement : vérifier `df.dtypes`, puis convertir avec `astype` (ou `to_numeric` pour les valeurs douteuses, `to_datetime` pour les dates).

Les conversions de type essentielles :

Opération	Ce qu'elle fait
<code>df["c"].astype(int)</code>	Convertit une colonne en entier.
<code>df["c"].astype(float)</code>	Convertit en nombre à virgule.
<code>df["c"].astype(str)</code>	Convertit en texte.
<code>df["c"].astype(bool)</code>	Convertit en booléen.
<code>df.astype({"a": int, "b": float})</code>	Convertit plusieurs colonnes d'un coup.
<code>pd.to_numeric(col, errors="coerce")</code>	Convertit en nombre ; met NaN sur les valeurs illisibles.
<code>col.str.replace(",", ".").astype(float)</code>	Corrige une virgule décimale puis convertit en float.
<code>pd.to_datetime(col)</code>	Convertit en dates (voir la sous-section sur les dates).
<code>df.dtypes</code>	Affiche le type de chaque colonne.

2.4 Filtrer et grouper

```
# Filtrer : tous les ordres sur ES
df[df["symbole"] == "ES"]
```

```
# Grouper : volume total par symbole
df.groupby("symbole")["volume"].sum()
# -> ES: 2700, NQ: 800, CL: 600
```

```
# Trier par prix décroissant
df.sort_values("prix", ascending=False)
```

Le **filtrage** `df[df["symbole"] == "ES"]` garde les lignes correspondant à une condition. Le **groupby** regroupe les lignes par valeur (ici par symbole) puis applique un calcul (la somme des volumes) — l'équivalent d'un tableau croisé dynamique. Ces deux opérations, filtrer et grouper, sont le pain quotidien de l'analyse de données de marché.

Pourquoi ça compte. Réconcilier des positions, agréger des volumes par symbole, calculer un PnL par stratégie : tout cela se fait en quelques lignes de Pandas. C'est l'outil central pour le reporting et l'analyse mentionnés dans l'offre Tower.

2.5 Les séries temporelles

Pandas excelle sur les **séries temporelles**, indispensables en trading. On peut indexer par date et rééchantillonner (changer la fréquence).

```
# Index temporel : chaque ligne est datée
df = pd.DataFrame({"prix": [5230, 5231, 5232, 5233]},
                  index=pd.to_datetime([
                      "2026-06-14 09:30", "2026-06-14 09:31",
                      "2026-06-14 09:32", "2026-06-14 09:33"]))

# Rééchantillonner : passer de la minute à la barre de 5 minutes
df["prix"].resample("5min").ohlc()  # open/high/low/close

# Moyenne mobile sur 3 périodes
df["prix"].rolling(3).mean()
```

Avec un index de dates, `resample("5min")` regroupe les données par tranches de 5 minutes — par exemple pour transformer des ticks en barres OHLC (ouverture, haut, bas, clôture). `rolling(3).mean()` calcule une moyenne mobile sur 3 périodes, glissant le long de la série. Ce sont les opérations fondamentales pour construire des indicateurs et agréger des données de marché.

Pourquoi ça compte. Les données de marché sont par nature temporelles. Le rééchantillonnage et les fenêtres glissantes de Pandas permettent de passer des ticks aux barres et de calculer des indicateurs en quelques lignes — directement utile pour ton bot.

2.6 Les dates et les formats de date

Les dates sont la principale source de problèmes lorsqu'on charge un fichier de marché : chaque source les écrit à sa façon, et un format mal interprété fausse tout le tri et toute l'analyse temporelle. Avant tout traitement chronologique (resample, rolling), il faut **convertir les dates en vraies dates** — des objets `datetime` — et non les laisser sous forme de simple texte.

Le problème : des dates en texte

```
df = pd.read_csv("ticks.csv")
df["date"]          # dtype: object -> ce sont des CHAÎNES, pas des dates !
df["date"].dt       # AttributeError : on ne peut pas trier/resampler du texte
```

À la lecture d'un CSV, Pandas laisse souvent les dates comme du **texte** (object). Trié comme du texte, « 2026-06-10 » passerait après « 2026-06-09 » mais « 10/06/2026 » serait trié sur le « 1 » : d'où des résultats faux. La première étape est donc de convertir.

La solution : `pd.to_datetime`

```
# Conversion simple (Pandas devine le format pour les formats standards)
df["date"] = pd.to_datetime(df["date"])
```

```
# Ou directement à la lecture du fichier
df = pd.read_csv("ticks.csv", parse_dates=["date"])
```

`pd.to_datetime` transforme une colonne de texte en vraies dates. Pour les formats standards (ISO « 2026-06-14 », par exemple), Pandas devine seul. Une fois converties, les dates se trient correctement et débloquent `.dt`, `resample` et `rolling`.

Dates mal écrites : préciser le format

Quand le fichier source a un format ambigu ou non standard, il faut le **préciser explicitement** avec `format=`, à l'aide des codes de format (voir le tableau plus bas). C'est le cas le plus fréquent en pratique.

```
# Format jour/mois/année (ambigu : 03/04 = 3 avril ou 4 mars ?)
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y") # 14/06/2026
```

```
# Format avec heure
df["date"] = pd.to_datetime(df["date"], format="%Y-%m-%d %H:%M:%S")
```

```
# Format "à l'américaine" mois/jour/année
df["date"] = pd.to_datetime(df["date"], format="%m/%d/%Y") # 06/14/2026
```

Préciser `format=` lève toute ambiguïté : « 03/04/2026 » devient sans équivoque le 3 avril (avec `%d/%m/%Y`) ou le 4 mars (avec `%m/%d/%Y`). C'est indispensable pour les dates « à l'européenne » (jour en premier) que Pandas interprète parfois à l'envers. Préciser le format rend aussi la conversion plus rapide et plus sûre.

Dates invalides ou manquantes : errors et coerce

```
# Si certaines lignes ont une date illisible, ne pas tout faire planter :
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y", errors="coerce")
# -> les dates illisibles deviennent NaT (Not a Time), au lieu de lever une erreur
```

```
# On peut ensuite repérer ou retirer ces lignes
df[df["date"].isna()] # les lignes à la date invalide
df = df.dropna(subset=["date"]) # les retirer
```

Avec `errors="coerce"`, une date illisible n'interrompt pas tout le chargement : elle est remplacée par NaT (l'équivalent de NaN pour les dates). On peut alors les repérer avec `isna()` et décider de les corriger ou de les écarter. C'est exactement la situation d'un fichier source aux dates mal écrites : on isole les lignes problématiques au lieu de perdre tout le jeu de données.

Date scindée en deux colonnes (en-tête décalé)

Cas fréquent et piégeux : le fichier a un en-tête de **8 colonnes** (dont une seule « date »), mais les **données** comptent **9 colonnes**, car la date y est écrite en deux morceaux séparés (par exemple la date et l'heure séparées par un espace traité comme un séparateur, « 2026-06-14 09:30 » devenant deux champs). Le nombre de colonnes de données ne correspond alors plus à l'en-tête, et la lecture se décale.

```
# Symptôme : l'en-tête a 8 noms, mais chaque ligne de données a 9 valeurs.  
# Souvent, Pandas met alors la 1re colonne (la date scindée) comme index,  
# ou décale toutes les colonnes d'un cran.
```

```
# Solution 1 : lire SANS en-tête, fixer soi-même les 9 vrais noms,  
#             puis recombinaison des deux morceaux de date.  
noms = ["date_j", "date_h", "symbole", "prix", "quantite",  
        "cote", "volume", "ouverture", "cloture"] # 9 colonnes réelles  
df = pd.read_csv("ticks.csv", header=0, names=noms)
```

```
# Recombiner les deux morceaux en une seule vraie date  
df["date"] = pd.to_datetime(df["date_j"] + " " + df["date_h"],  
                             format="%Y-%m-%d %H:%M:%S")  
df = df.drop(columns=["date_j", "date_h"]) # retirer les deux morceaux
```

On lit le fichier en imposant les **9 vrais noms de colonnes** (names=), ce qui réaligne les données. Puis on **recolle** les deux morceaux de date en les concaténant comme du texte (avec un espace entre les deux) avant de les passer à `pd.to_datetime`. Enfin, on supprime les deux colonnes intermédiaires. Le point clé : `df["date_j"] + " " + df["date_h"]` reconstitue la chaîne complète « 2026-06-14 09:30:00 », que `to_datetime` convertit ensuite en vraie date.

Une autre approche, si le séparateur fautif est bien identifié (un espace là où il faudrait garder la date entière), consiste à forcer le bon séparateur de colonnes à la lecture :

```
# Solution 2 : si les colonnes sont séparées par des virgules mais que la  
#             date contient un espace, ce n'est PAS le séparateur de colonnes.  
#             On lit avec le bon séparateur, et la date reste entière.  
df = pd.read_csv("ticks.csv", sep=",") # l'espace dans la date est préservé  
df["date"] = pd.to_datetime(df["date"])
```

Le vrai diagnostic est donc : **l'espace dans la date a été pris pour un séparateur de colonnes**. Si le fichier utilise réellement la virgule comme séparateur, préciser `sep=","` suffit à garder la date en un seul champ. Si la date est bien stockée en deux colonnes distinctes dans le fichier, la solution 1 (recombinaison) est la bonne. Dans tous les cas, on inspecte d'abord le fichier brut (par exemple `df.head()` et `df.shape`) pour comprendre le décalage avant de corriger.

Pourquoi ça compte. Un décalage entre le nombre de colonnes de l'en-tête et celui des données est presque toujours dû à un séparateur mal interprété, souvent dans la date. Le réflexe : regarder les premières lignes brutes, compter les colonnes réelles, puis soit fixer le bon séparateur (`sep=`), soit imposer les vrais noms (`names=`) et recombinaison des morceaux. C'est typique des relevés de courtiers aux formats hétérogènes.

Extraire et manipuler des composantes de date

```
# L'accesseur .dt donne accès aux composantes
df["date"].dt.year          # l'année
df["date"].dt.hour         # l'heure
df["date"].dt.day_name()    # le jour de la semaine ("Monday"...)
```

```
df["date"].dt.date          # juste la partie date, sans l'heure
```

```
# Reformater une date en texte (pour un rapport, un nom de fichier)
```

```
df["date"].dt.strftime("%Y-%m-%d")    # -> "2026-06-14"
```

Une fois les dates converties, l'accesseur `.dt` donne accès à toutes leurs composantes (année, heure, jour de la semaine...), utile pour filtrer (par exemple ne garder que les heures d'ouverture du marché) ou agréger. `strftime` fait l'inverse de `to_datetime` : il reformate une date en texte selon un motif choisi, pratique pour un rapport ou un nom de fichier.

Lien avec les fuseaux horaires (section 3)

```
# Rendre des dates "avisées" (avec fuseau) après conversion
```

```
df["date"] = df["date"].dt.tz_localize("America/New_York")
```

```
# Convertir vers un autre fuseau
```

```
df["date"] = df["date"].dt.tz_convert("America/Montreal")
```

Une fois les dates converties, on peut leur attribuer un fuseau avec `tz_localize` (les rendre « avisées », voir la section 3) puis les convertir avec `tz_convert`. C'est l'équivalent, sur une colonne entière, du `astimezone` vu pour une date isolée.

Pourquoi ça compte. Sur un fichier de marché, la conversion des dates est presque toujours la première étape, et la plus piégeuse. Préciser `format=` pour les dates ambiguës, et `errors="coerce"` pour ne pas planter sur quelques lignes corrompues, évite des erreurs silencieuses qui décaleraient tout le tri temporel — un classique de la réconciliation entre sources aux formats divergents.

Les codes de format de date les plus courants :

Code	Signification (exemple pour le 14 juin 2026, 9h30)
%Y	Année sur 4 chiffres (2026).
%y	Année sur 2 chiffres (26).
%m	Mois sur 2 chiffres (06).
%B	Le code pour un nom de mois complet (juin)
%d	Jour sur 2 chiffres (14).
%H	Heure sur 24 h (09).

Code	Signification (exemple pour le 14 juin 2026, 9h30)
%M	Minutes (30).
%S	Secondes (00).
%d/%m/%Y	Format européen : 14/06/2026.
%m/%d/%Y	Format américain : 06/14/2026.
%Y-%m-%d	Format ISO (recommandé) : 2026-06-14.
%Y-%m-%d %H:%M:%S	Date et heure complètes : 2026-06-14 09:30:00.

Les opérations de date essentielles :

Opération	Ce qu'elle fait
<code>pd.to_datetime(col)</code>	Convertit du texte en vraies dates (devine les formats standards).
<code>pd.to_datetime(col, format="...")</code>	Convertit en précisant le format exact (dates ambiguës ou non standards).
<code>errors="coerce"</code>	Met NaT sur les dates illisibles au lieu de planter.
<code>parse_dates=["col"]</code>	Convertit les dates dès la lecture du CSV (<code>read_csv</code>).
<code>pd.read_csv(..., names=[...])</code>	Impose les vrais noms de colonnes (en-tête décalé / date scindée).
<code>to_datetime(col_j + " " + col_h)</code>	Recombine une date scindée en deux colonnes avant conversion.
<code>.dt.year / .dt.hour / ...</code>	Extrait une composante (année, heure, jour...).
<code>.dt.day_name()</code>	Donne le jour de la semaine.
<code>.dt.strftime("...")</code>	Reformate une date en texte selon un motif.
<code>.dt.tz_localize("...")</code>	Attribue un fuseau (rend la date « avisée »).
<code>.dt.tz_convert("...")</code>	Convertit la colonne vers un autre fuseau.

Récapitulatif

1. **NumPy** : la vectorisation applique une opération à tout un tableau d'un coup, en code C rapide ; éviter les boucles Python. Le code natif relâche le GIL et peut, pour les calculs lourds, utiliser plusieurs cœurs.
2. **Pandas DataFrame** : un tableau à colonnes nommées ; charger (`read_csv`), filtrer, grouper, trier en quelques lignes.
3. **Séries temporelles** : indexer par date, rééchantillonner (`resample`), fenêtres glissantes (`rolling`).
4. **Le temps** : toujours des dates avisées (avec fuseau) ; convertir avec `astimezone` pour des marchés mondiaux.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur NumPy

Q1. Qu'est-ce que la vectorisation et pourquoi est-elle rapide ?

Réponse. Appliquer une opération à tout un tableau d'un coup, sans boucle Python. C'est rapide parce que l'opération s'exécute en code C optimisé sous le capot, bien plus vite qu'une boucle Python.

Q2. La vectorisation libère-t-elle le GIL, et est-ce la raison de sa vitesse ?

Réponse. Le code natif appelé par NumPy relâche effectivement le GIL, ce qui permet aux opérations lourdes (comme `A @ B`) d'utiliser plusieurs cœurs. Mais la source principale du gain est ailleurs : éviter l'interpréteur Python (code C, même sur un seul cœur). Les deux effets se cumulent, sans être identiques.

Q3. Quelle est la règle d'or de NumPy ?

Réponse. Éviter les boucles Python : penser en opérations sur des tableaux entiers plutôt qu'élément par élément.

Q4. Que fait `prix[prix > 5232]` ?

Réponse. C'est un filtrage booléen : il renvoie tous les éléments du tableau supérieurs à 5232, sans boucle ni condition explicite.

Sur Pandas

Q5. Qu'est-ce qu'un DataFrame ?

Réponse. Un tableau à colonnes nommées, comme une feuille de calcul ou une table de base de données ; l'outil de référence pour manipuler des données structurées.

Q6. Comment charge-t-on un fichier dans un DataFrame ?

Réponse. Avec `pd.read_csv` pour un CSV (ou `pd.read_parquet` pour un Parquet). On peut préciser `parse_dates` et `index_col` pour obtenir directement un index temporel.

Q7. À quoi sert `groupby` ?

Réponse. À regrouper les lignes par valeur d'une colonne (par exemple par symbole) puis à appliquer un calcul (somme, moyenne...) à chaque groupe. C'est l'équivalent d'un tableau croisé dynamique.

Q8. À quoi servent `resample` et `rolling` sur une série temporelle ?

Réponse. `resample` change la fréquence des données (par exemple regrouper des minutes en barres de 5 minutes). `rolling` calcule sur une fenêtre glissante (par exemple une moyenne mobile).

Sur la gestion du temps

Q9. Quelle est la différence entre une date naïve et une date avisée ?

Réponse. Une date naïve n'a pas de fuseau horaire : elle est ambiguë. Une date avisée inclut un fuseau (`tzinfo`), donc désigne un instant sans ambiguïté.

Q10. Pourquoi la gestion des fuseaux est-elle critique en trading ?

Réponse. Parce que les marchés sont dans des fuseaux différents. Une erreur de fuseau peut décaler les données de plusieurs heures et fausser des signaux ou des réconciliations.

Q11. Comment convertit-on un instant d'un fuseau à un autre ?

Réponse. Avec `astimezone`, qui change le fuseau tout en préservant le moment réel (par exemple convertir une heure de New York vers Montréal ou Londres).

Document suivant (6/8) : l'interaction avec le monde extérieur — requêtes HTTP, WebSockets pour le temps réel, JSON, bases de données et fichiers.